

Homework 5: All about Files

PART I: Basic File I/O

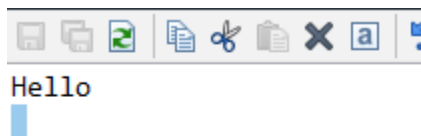
8 points (2 pts each)

Write a Program to create a text file for writing. Use fprintf() to write a simple text message such as "Hello\n" to the file and close it. Now re-open it in read mode and determine the following:

- (a) Whether or not the '\n' character is translated into a line feed sequence. .
Yes, the \n character is translated.

```
#include <stdio.h>

int main (void) {
    FILE *fl;
    fl = fopen("tester.txt", "w");
    fprintf(fl, "Hello\n");
    int ret;
    ret = fclose(fl);
}
```



Close the file. Next, open the file in append mode.

- (b) Determine whether the SPI is positioned at the start or end of the file when an existing file is opened in append mode. You can use ftell()/fseek() for that;

SPI is at end of file.

```
jblazina@cscd-linux01:~/CSCD240/HW5$ ./Append

SPI is at : 6
jblazina@cscd-linux01:~/CSCD240/HW5$
```

- (c) In append mode, try to use fseek() to move SPI at the beginning. Then write "Goodbye CSCD240\n" in the file. Does it write at the beginning or at the end?

It writes it at the beginning of the file.

```
Goodbye CSCD240Hello

#include <fcntl.h>
#include <stdio.h>

int main (void) {
    FILE *fl;
    fl = fopen("tester.txt", "a");
    fprintf(fl, "Hello\n");
    fseek(fl, 0L, 0);
    fprintf(fl, "Goodbye CSCD240");
    int ret = ftell(fl);
    printf("\n SPI is at : %d\n",ret);
}
```

- (d) Whether the operating system permits empty files of zero length to remain in existence after they have been closed.

Yes it seems to permit it

```
linux01:~/CSCD240/HW5$ ./Ap
linux01:~/CSCD240/HW5$ ls
c  does_this_close.txt  HW
```

```
#include <fcntl.h>
#include <stdio.h>

int main (void) {
    FILE *f1;
    f1 = fopen("does_this_close.txt", "a");
    // fprintf(f1, "Hello\n");
    // fseek(f1, 0L, 0);
    // fprintf(f1, "Goodbye CSCD240");
    // int ret = ftell(f1);
    // printf("\n SPI is at : %d\n",ret);
    int ret = fclose(f1);
}
```

Answer (a) to (d) with code snippets and screenshots in a pdf file named **HW5part1.pdf**.

PART II: String Match and Empty Line Count in a Text File

8 points (4 + 4)

OUTPUT:

```
jblazina@cscd-linux01: ~/CSCD240/HW5/Part2
jblazina@cscd-linux01:~/CSCD240/HW5/Part2$ ./file poem.txt RAGE
The string "RAGE" occurs 8 times in File "poem.txt".
File "poem.txt" has 5 empty lines.
jblazina@cscd-linux01:~/CSCD240/HW5/Part2$ ./file poem.txt rage
The string "rage" occurs 8 times in File "poem.txt".
File "poem.txt" has 5 empty lines.
jblazina@cscd-linux01:~/CSCD240/HW5/Part2$ ./file poem.txt RaGe
The string "RaGe" occurs 8 times in File "poem.txt".
File "poem.txt" has 5 empty lines.
jblazina@cscd-linux01:~/CSCD240/HW5/Part2$
```

Makefile: For PARTII, you need to use **Makefile**; split the attached file **file.c** into three files: (a) header file named **fileTest.h** and (b) two separate source files: **fileTest.c** and (c) **file.c**. **fileTest.h** will only contain function declaration. For convenience, you can use additional functions. **fileTest.c** will contain function body. **file.c** will include the main function only.

4 pts

Submission:

Your final submission will be a **zip** file that will include the following files:

fileTest.h fileTest.c.....

file.c and

Makefile

HW5part1.pdf

Name your zip file with your last name first letter of your first name HW5.zip (ex: yasminsHW5.zip).

Submission deadline is: 11:59 pm, Monday, March 18. No late submission will be considered.